

AT32F435/437 Performance Improvement

Introduction

This application note introduces how to use software to improve performance of AT32F435_437 MCUs.

Applicable products:

Part number	AT32F435 series
	AT32F437 series

Contents

1	Overview	5
2	Expand SRAM and ZW area	6
2.1	Requirement analysis and trade-offs	6
2.2	How to expand and configure	7
3	Use SRAM1 and SRAM2.....	9
3.1	Independent use of SRAM1/SRAM2	9
3.2	SRAM1 mapping	11
4	Enable FLASH continuous read mode	13
4.1	How to enable Flash continuous read mode.....	13
5	Revision history.....	14

List of tables

Table 1. Extended system options.....	7
Table 2. Document revision history.....	14

List of figures

Figure 1. SRAM default configuration and extension modes	6
Figure 2. Use ICP tool to configure SRAM	8
Figure 3. AT32F435/437 address mapping	9
Figure 4. AHB bus matrix.....	10
Figure 5. Use SRAM1 or SRAM2	11
Figure 6. Use both SRAM1 and SRAM2	11
Figure 7. Bus address range	12
Figure 8. Access variable A (SRAM1 address 0x2000_0000)	12
Figure 9. Access variable A (SRAM1 address mapping 0x1000_0000)	12
Figure 10. Read access time with Flash continuous access mode disabled.....	13
Figure 11. Read access time with Flash continuous access mode enabled.....	13

1 Overview

The improvement of system performance is realized by adjusting and enhancing various features and functions. Therefore, before starting performance improvement, the user should be clear about the structure and parameters of hardware (e.g., Flash size, SRAM size, zero-wait (ZW) and non-zero-wait (NZW) area) and software (be familiar with the whole process, such as the execution time of codes and algorithms, and frequency of access to important data); meanwhile, the user should optimize the system according to the actual project, so as to improve performance.

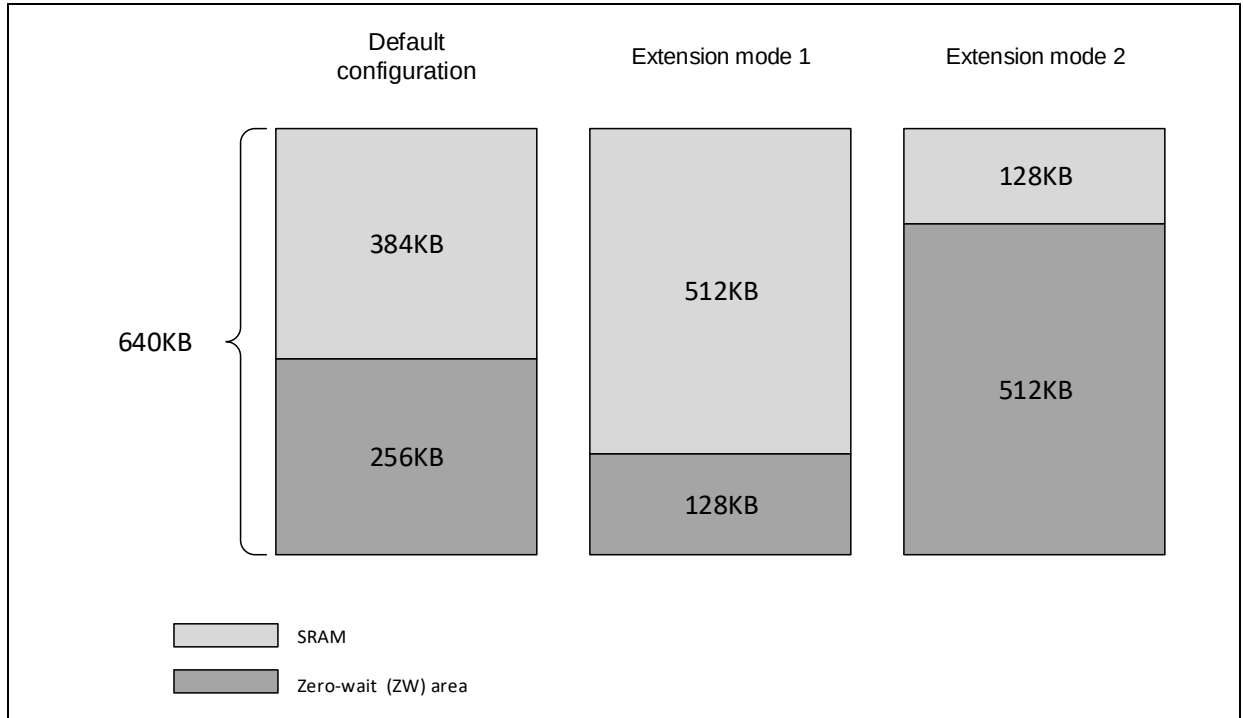
This application note introduce how to improve AT32F435/437 series performance with the following methods:

- Expand SRAM and ZW area
- Use SRAM1 and SRAM2
- Enable FLASH continuous read mode

2 Expand SRAM and ZW area

The AT32F435/437 series MCUs contain a 384 KB on-chip SRAM and 256 KB ZW, by default. Users can re-configure the SRAM and ZW area size as needed. Figure 1 shows the default configuration, maximum SRAM and maximum ZW area configurations.

Figure 1. SRAM default configuration and extension modes



2.1 Requirement analysis and trade-offs

- The zero-wait state (ZW) area is an internal preloaded storage area between FLASH and CPU for high-speed operation, which is essentially a RAM. After the chip is powered on, the hardware automatically copies ZW size from the Flash start address to the ZW area, realizing the fast access to the preloaded data during the subsequent system operation. The program can realize better performance running in zero-wait area than non-zero-wait area.
- SRAM is the storage area when system is running. It stores instructions, data and stacks, which is necessary for normal operation of programs.

Since the total size of ZW area and SRAM is limited to 640 KB, the user should configure and make reasonable trade-offs according to the actual application.

The recommended configuration and extension is as follows:

1. After completion of system compilation, check the required RAM. If the RAM size is larger than the default SRAM, the user needs to increase SRAM and reduce ZW area size to ensure that the re-configured SRAM is larger than the required size for normal operation.
2. After completion of system compilation, check the required ROM. If the ROM size is larger than the default ZW area and it stores a lot of data to be called frequently or functions requiring high performance,

- Assign the address of frequently called time or the function requiring high performance to the ZW area by modifying the scatter-loading file or specifying the address directly in code;
- If the requirement is still not met after the above operation, check the required RAM. If the RAM size is smaller than the default SRAM and is still enough after reducing SRAM size, the user can increase ZW area and reduce SRAM to allow the ZW area to contain as much code as possible.

2.2 How to expand and configure

The extended system option EOPB0 bit is in the user system data (USD) area in the Flash, which supports extension in 64 KB, as shown in Table 1.

Table 1. Extended system options

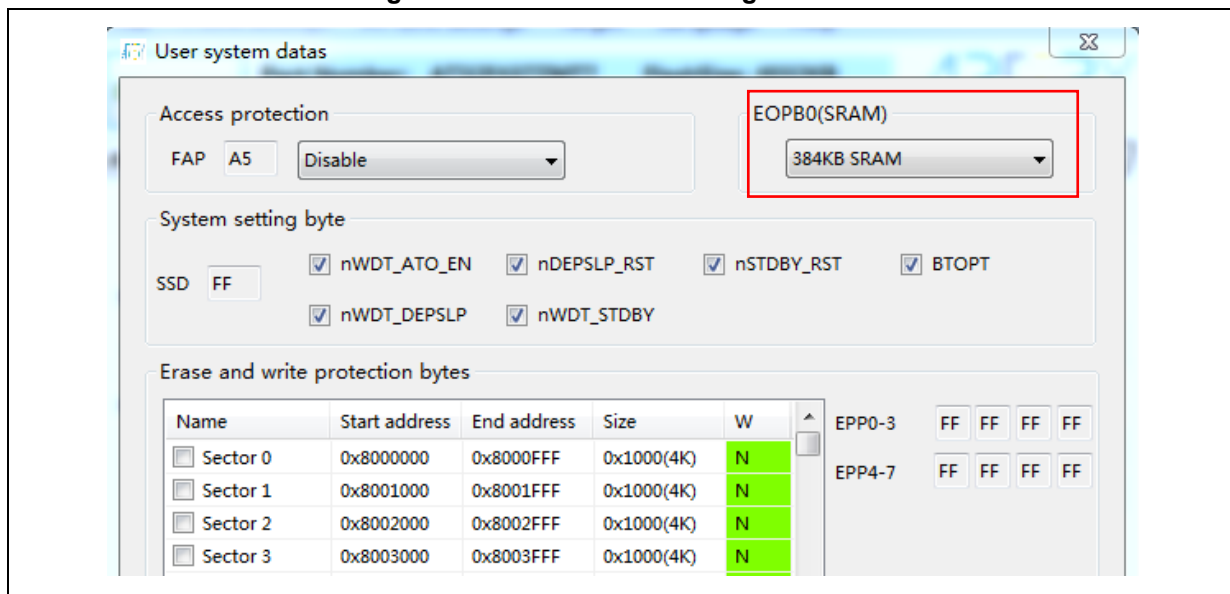
EOPB0[7:0]: Extended system option		
256 K Flash	Bit 1:0	00: On-chip 512 KB SRAM + 128 KB zero-wait-state Flash 01: On-chip 448 KB SRAM + 192 KB zero-wait-state Flash 10, 11: On-chip 384 KB SRAM + 256 KB zero-wait-state Flash Note: Bit 1~0 can be altered only when security library is disabled.
	Bit 7:2	Reserved
1024 K and above Flash	Bit 2:0	000: On-chip 512 KB SRAM + 128 KB zero-wait-state Flash 001: On-chip 448 KB SRAM + 192 KB zero-wait-state Flash 010: On-chip 384 KB SRAM + 256 KB zero-wait-state Flash 011: On-chip 320 KB SRAM + 320 KB zero-wait-state Flash 100: On-chip SRAM 256 KB SRAM + 384 KB zero-wait-state Flash 101: On-chip SRAM 192 KB SRAM + 448 KB zero-wait-state Flash 110, 111: On-chip 128 KB SRAM + 512 KB zero-wait-state Flash Note: Bit 2~0 can be altered only when security library is disabled.
	Bit 7:3	Reserved

The SRAM and ZW area can be extended with the following methods:

- In mass production phase, it is recommended to use programming tools to program the configured user system data (USD) and code;
- In product development phase, it is recommended to use ICP tool or user program code for modification. Refer to the document FAQ0054_how_to_modify_SRAM_size_EN.

For details about extended configuration in user program code, refer to *extend_sramin* in BSP and *AN0026_Extending_SRAM_in_User's_Program.pdf*. Figure 2 is the configuration UI using ICP tool.

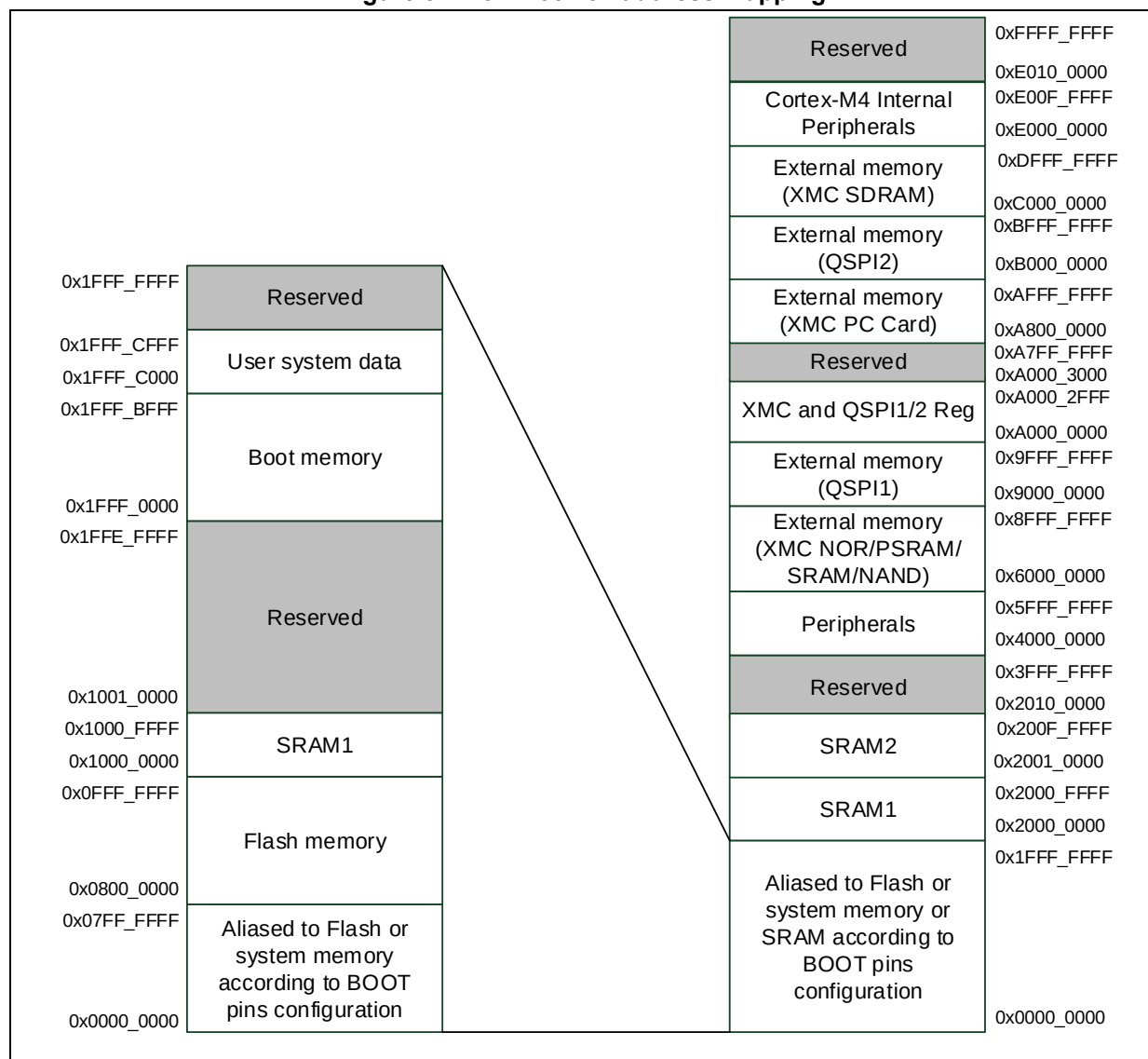
Figure 2. Use ICP tool to configure SRAM



3 Use SRAM1 and SRAM2

The on-chip SRAM (128 KB~512 KB) of AT32F435/437 series is divided into SRAM1 and SRAM2. The SRAM1 is fixed to 64 KB with the address of 0x2000_0000~0x2000_FFFF, which can be mapped to 0x1000_0000~0x1000_FFFF. The SRAM2 size ranges from 64 KB to 448 KB (depending on the setting of EOPB0 bit), with the start address of 0x2001_0000. Their respective address mappings are shown in Figure 3.

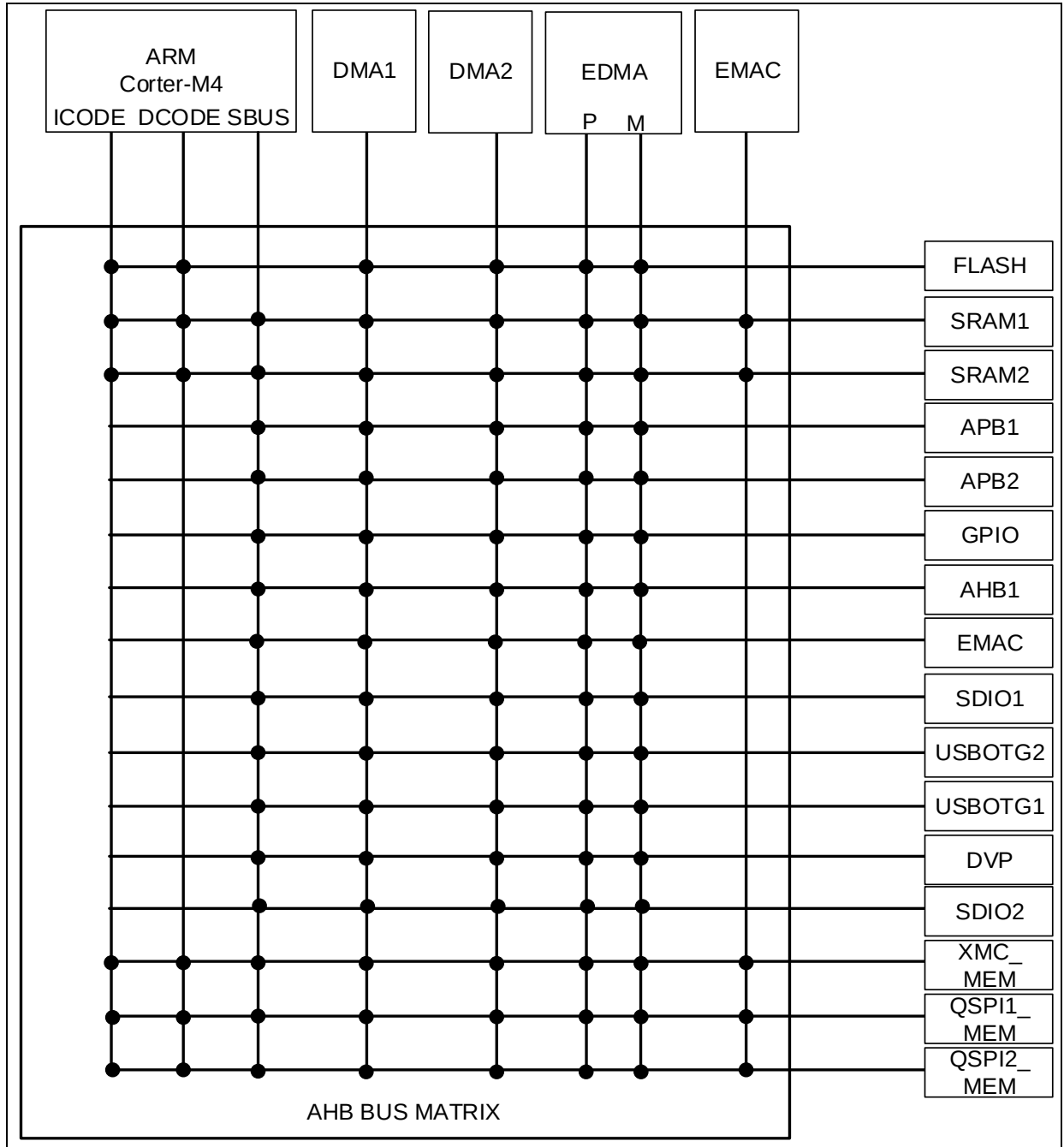
Figure 3. AT32F435/437 address mapping



3.1 Independent use of SRAM1/SRAM2

The SRAM1 and SRAM2 can be regarded as two independent RAM. As shown in Figure 4, the SRAM1 and SRAM2 can be accessed independently, which improves the Flash access speed in some scenarios.

Figure 4. AHB bus matrix



When it is necessary to load data from Flash to the specified memory quickly during system operation, for example, using DMA to load data from Flash to SRAM:

- In case of conventional design with only one SRAM, there is a conflict when the Flash is accessed by any program other than DMA, resulting in a lower efficiency.
- In case of design of independent SRAM1 and SRAM2, the DMA load data from Flash to SRAM1 and the other programs use SRAM2, which can be executed simultaneously without conflict, thus improving efficiency.

A simple test program is designed for validation. Perform looping execution of IO toggling and variable (SRAM2) increment; at the same time, DMA loads FLASH data to SRAM1. The comparison of IO toggling frequencies shows that when the memory and program variable handled

by DMA are in the SRAM1 and SRAM2 respectively, the IO toggling frequency is 377KHz, which is larger than 347 KHz (the frequency when both memory and program variable handled by DMA are in the same RAM (SRAM1 or SRAM2)). The IO toggling frequencies in the test program are shown in Figure 5 and Figure 6.

Figure 5. Use SRAM1 or SRAM2

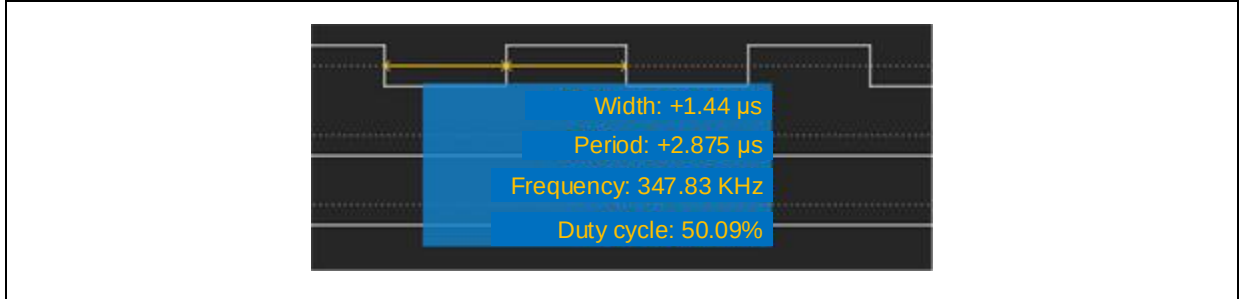
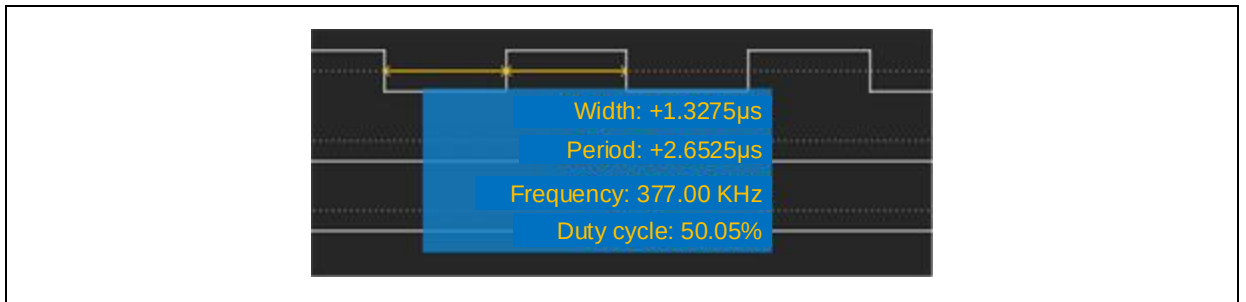


Figure 6. Use both SRAM1 and SRAM2



3.2 SRAM1 mapping

The Cortex-M4 Technical Reference Manual describes that the address ranges of I-BUS and D-BUS in the system are both below 0x2000_000, and the SRAM1 address range is from 0x2000_0000 to 0x2000_FFFF, which can be mapped to 0x1000_0000 to 0x1000_FFFF. That is, SRAM1 can be accessed through I-BUS, D-BUS or S-BUS, which improves the flexibility and improves Flash access speed in some scenarios.

Figure 7. Bus address range

The ICode memory interface. Instruction fetches from Code memory space (0x00000000 - 0x1FFFFFFF) are performed over this 32-bit *Advanced High-performance Bus* (AHB)-Lite bus. For more information, see *ICode bus interface* on page 14-4.

The DCode memory interface. Data and debug accesses to Code memory space (0x00000000 - 0x1FFFFFFF) are performed over this 32-bit AHB-Lite bus. For more information, see *DCode bus interface* on page 14-6.

The System interface. Instruction fetches, and data and debug accesses, to System space (0x20000000 - 0xDFFFFFFF, 0xE0100000 - 0xFFFFFFFF) are performed over this 32-bit AHB-Lite bus. For more information, see *System interface* on page 14-7.

For example, when accessing multiple variables in the system:

- If SRAM1 is at the address of 0x2000_0000, these variables only can be accessed through S-BUS sequentially;
- If SRAM1 can be mapped to 0x1000_0000, these variables can be accessed through S-BUS and D-BUS respectively, which improves efficiency.

A simple test program is designed for validation. Perform looping execution of IO toggling and increment of variable A (in SRAM1) and variable B (in SRAM2). The IO toggling frequency is 2.06 MHz when accessing variable A (through SRAM1 address mapping 0x1000_0000), which is larger than 2.00 MHz (through SRAM1 address 0x2000_0000). The IO toggling frequencies in the test program are shown in Figure 8 and Figure 9.

Figure 8. Access variable A (SRAM1 address 0x2000_0000)

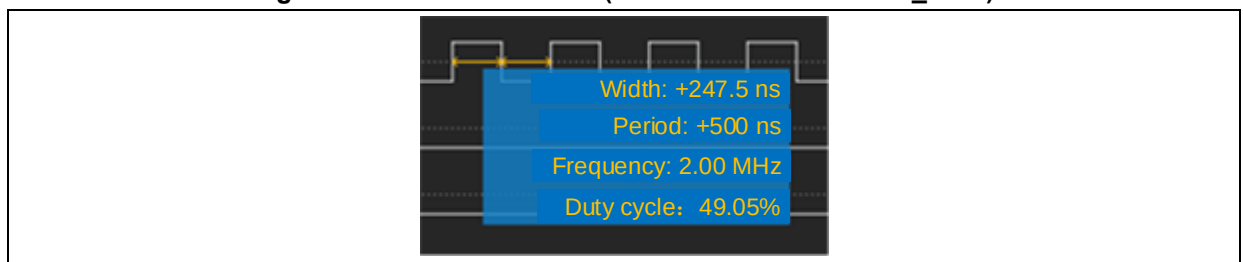
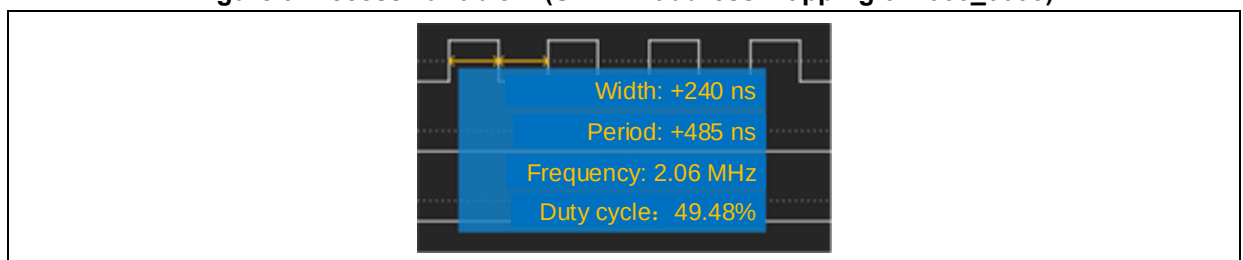


Figure 9. Access variable A (SRAM1 address mapping 0x1000_0000)



4 Enable FLASH continuous read mode

The Flash continuous access mode is enabled by setting FCONTR_EN=1 in the FLASH_CONTR register.

After enabling the Flash continuous read mode, the efficiency of reading bulk continuous addresses from the non-zero-wait state Flash is significantly improved.

Note: Enabling this mode increases the MCU operating power consumption by several milliamps during continuous addresses reading.

4.1 How to enable Flash continuous read mode

In the program initialization phase, call the `flash_continue_read_enable()` function in BSP and set "TRUE" to enable the Flash continuous read mode.

A simple test program is designed for validation. The program reads 32 KB of data continuously from a certain address in the non-zero-wait state Flash and loop for 1000 times, and record the read access time. Test result shows that the read access time is 2.231s when the Flash continuous access mode is enabled, which is 34% shorter than the read access time (3.369s when the Flash continuous access mode is disabled). The time between "1" and "2" (as shown in Figure 10 and Figure 11) is the recorded read access time.

Figure 10. Read access time with Flash continuous access mode disabled

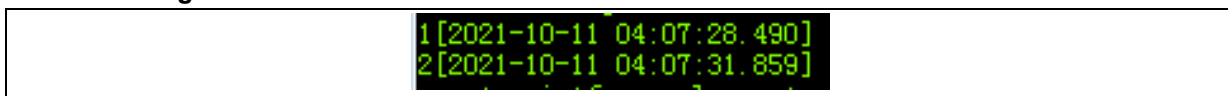
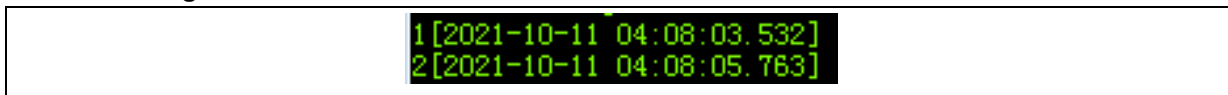


Figure 11. Read access time with Flash continuous access mode enabled



5 Revision history

Table 2. Document revision history

Date	Version	Revision note
2021.10.8	2.0.0	Initial release.
2026.1.27	2.0.1	Updated Section 2.2 by adding the document FAQ054_how_to_modify_SRAM_size as reference.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services, and ARTERY assumes no liability whatsoever relating to the choice, selection or use of the ARTERY products and services described herein

No license, express or implied, to any intellectual property rights is granted under this document. If any part of this document deals with any third party products or services, it shall not be deemed a license granted by ARTERY for the use of such third party products or services, or any intellectual property contained therein, or considered as a warranty regarding the use in any manner of such third party products or services or any intellectual property contained therein.

Unless otherwise specified in ARTERY's terms and conditions of sale, ARTERY provides no warranties, express or implied, regarding the use and/or sale of ARTERY products, including but not limited to any implied warranties of merchantability, fitness for a particular purpose (and their equivalents under the laws of any jurisdiction), or infringement on any patent, copyright or other intellectual property right.

Purchasers hereby agree that ARTERY's products are not designed or authorized for use in: (A) any application with special requirements of safety such as life support and active implantable device, or system with functional safety requirements; (B) any aircraft application; (C) any aerospace application or environment; (D) any weapon application, and/or (E) or other uses where the failure of the device or product could result in personal injury, death, property damage. Purchasers' unauthorized use of them in the aforementioned applications, even if with a written notice, is solely at purchasers' risk, and Purchasers are solely responsible for meeting all legal and regulatory requirements in such use.

Resale of ARTERY products with provisions different from the statements and/or technical characteristics stated in this document shall immediately void any warranty grant by ARTERY for ARTERY's products or services described herein and shall not create or expand any liability of ARTERY in any manner whatsoever.

© 2026 Artery Technology -All rights reserved